

Artificial Intelligence Lab - Lab 06

8-Queen Problem: Hill Climbing + Simulated Annealing

Name: Mehr Ali

Semester: 4B-BSCS

Problem Statement

Place 8 queens on an 8x8 chessboard such that **no two queens attack each other** — no two queens on the same row, column, or diagonal.

We solve this using two algorithms:

- **Hill Climbing** — always moves to the best neighbor, stops at local minimum
- **Simulated Annealing** — sometimes accepts worse moves to escape local minimums

The heuristic **h** = number of pairs of queens attacking each other. Goal is **h = 0**.

Step 1: Import Libraries

```
In [1]: import random
import math
import matplotlib.pyplot as plt
import matplotlib.patches as patches
```

Step 2: Random Board and Objective Function

```
In [2]: def random_board():
        return [random.randint(0, 7) for _ in range(8)]

def h(board):
    attacks = 0
    for i in range(8):
        for j in range(i+1, 8):
            if board[i] == board[j]:
                attacks += 1
            if abs(board[i] - board[j]) == abs(i - j):
                attacks += 1
    return attacks
```

Step 3: Draw Chessboard Function

```
In [3]: def draw_board(board, title='8-Queen Solution'):
    fig, ax = plt.subplots(figsize=(6, 6))
    for row in range(8):
        for col in range(8):
            color = 'white' if (row + col) % 2 == 0 else 'gray'
            rect = patches.Rectangle((col, row), 1, 1, linewidth=1, edgecolor='black')
            ax.add_patch(rect)
    for col in range(8):
        row = board[col]
        ax.text(col + 0.5, row + 0.5, '♔', fontsize=20, ha='center', va='center', color='black')
    ax.set_xlim(0, 8)
    ax.set_ylim(0, 8)
    ax.set_xticks(range(8))
    ax.set_yticks(range(8))
    ax.set_xticklabels(['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'])
    ax.set_yticklabels(range(1, 9))
    ax.set_title(title, fontsize=13)
    plt.tight_layout()
    plt.show()
```

Part 1: Hill Climbing

Step 4: Hill Climbing Function

```
In [4]: def hill_climbing():
    board = random_board()

    while True:
        current_h = h(board)

        if current_h == 0:
            return board

        best_board = None
        best_h = current_h

        for col in range(8):
            for row in range(8):
                if row == board[col]:
                    continue
                neighbor = board[:]
                neighbor[col] = row
                neighbor_h = h(neighbor)
                if neighbor_h < best_h:
                    best_h = neighbor_h
                    best_board = neighbor
```

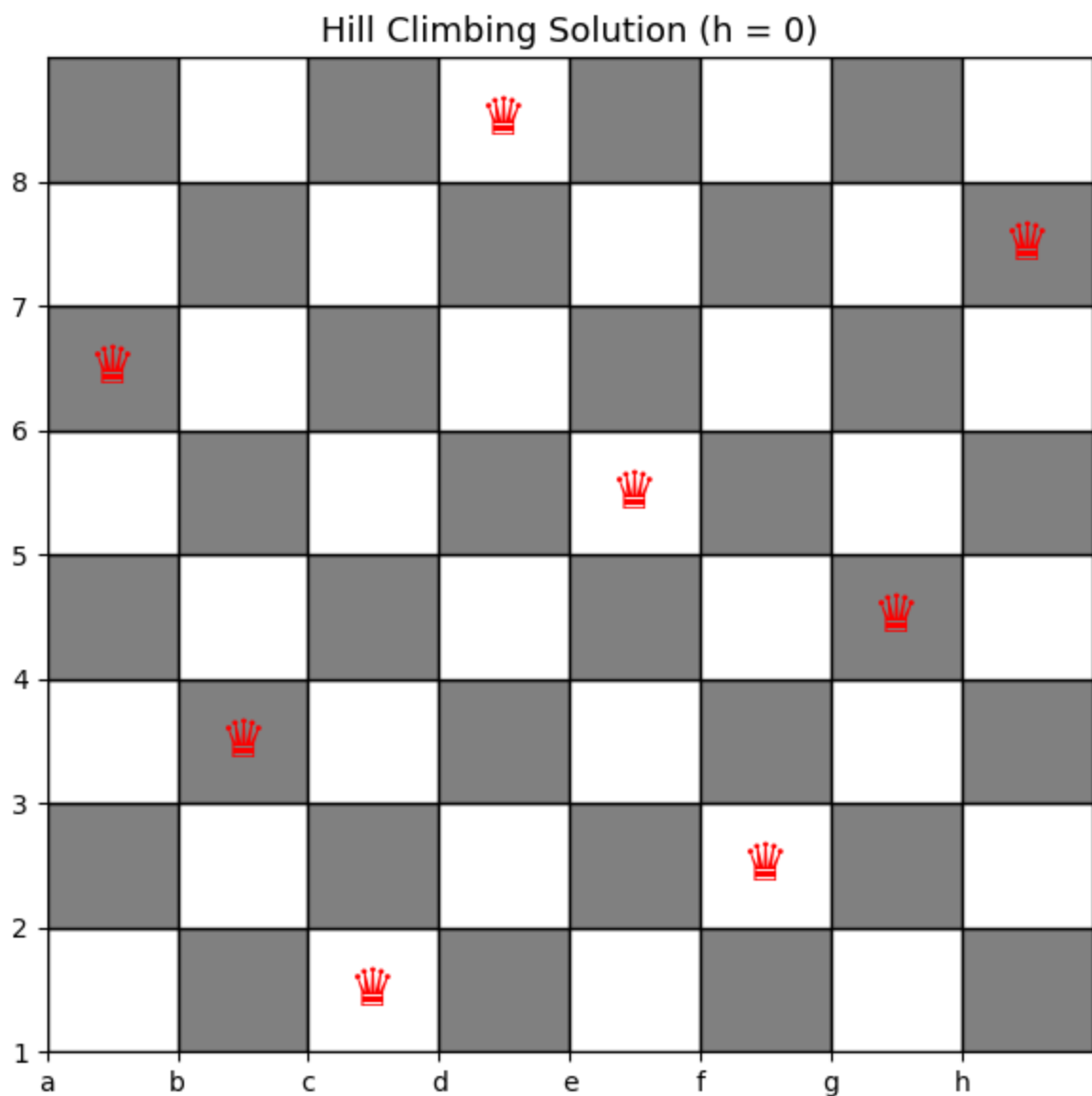
```
    if best_board is None:  
        return None  
  
    board = best_board
```

Step 5: Run Hill Climbing

```
In [5]: hc_solution = None  
        attempts = 0  
  
        while hc_solution is None:  
            attempts += 1  
            hc_solution = hill_climbing()  
  
        print("Hill Climbing Solution found after", attempts, "attempt(s)")  
        print("Board:", hc_solution)  
        print("h value:", h(hc_solution))
```

```
Hill Climbing Solution found after 1 attempt(s)  
Board: [5, 2, 0, 7, 4, 1, 3, 6]  
h value: 0
```

```
In [6]: draw_board(hc_solution, title=f'Hill Climbing Solution (h = {h(hc_solution)})')
```



Part 2: Simulated Annealing (Bonus Task)

Step 6: Simulated Annealing Function

```
In [7]: def simulated_annealing():  
        board = random_board()  
        current_h = h(board)  
  
        T = 100.0  
        cooling = 0.995  
  
        h_history = [current_h]  
  
        while T > 0.1:  
            if current_h == 0:
```

```
        break

    col = random.randint(0, 7)
    row = random.randint(0, 7)

    neighbor = board[:]
    neighbor[col] = row
    neighbor_h = h(neighbor)

    delta = neighbor_h - current_h

    if delta < 0:
        board = neighbor
        current_h = neighbor_h
    else:
        probability = math.exp(-delta / T)
        if random.random() < probability:
            board = neighbor
            current_h = neighbor_h

    T = T * cooling
    h_history.append(current_h)

    return board, h_history
```

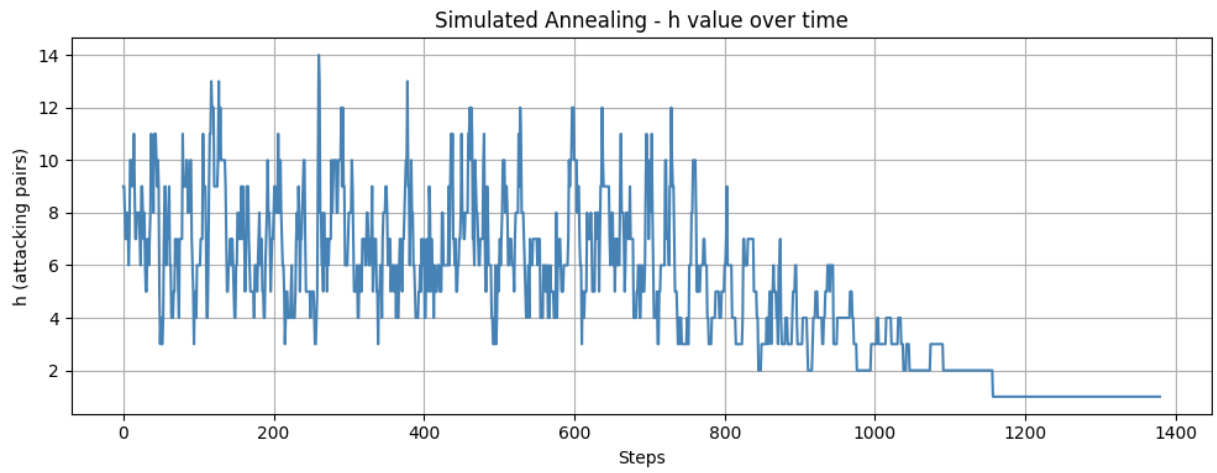
Step 7: Run Simulated Annealing

```
In [8]: sa_solution, h_history = simulated_annealing()

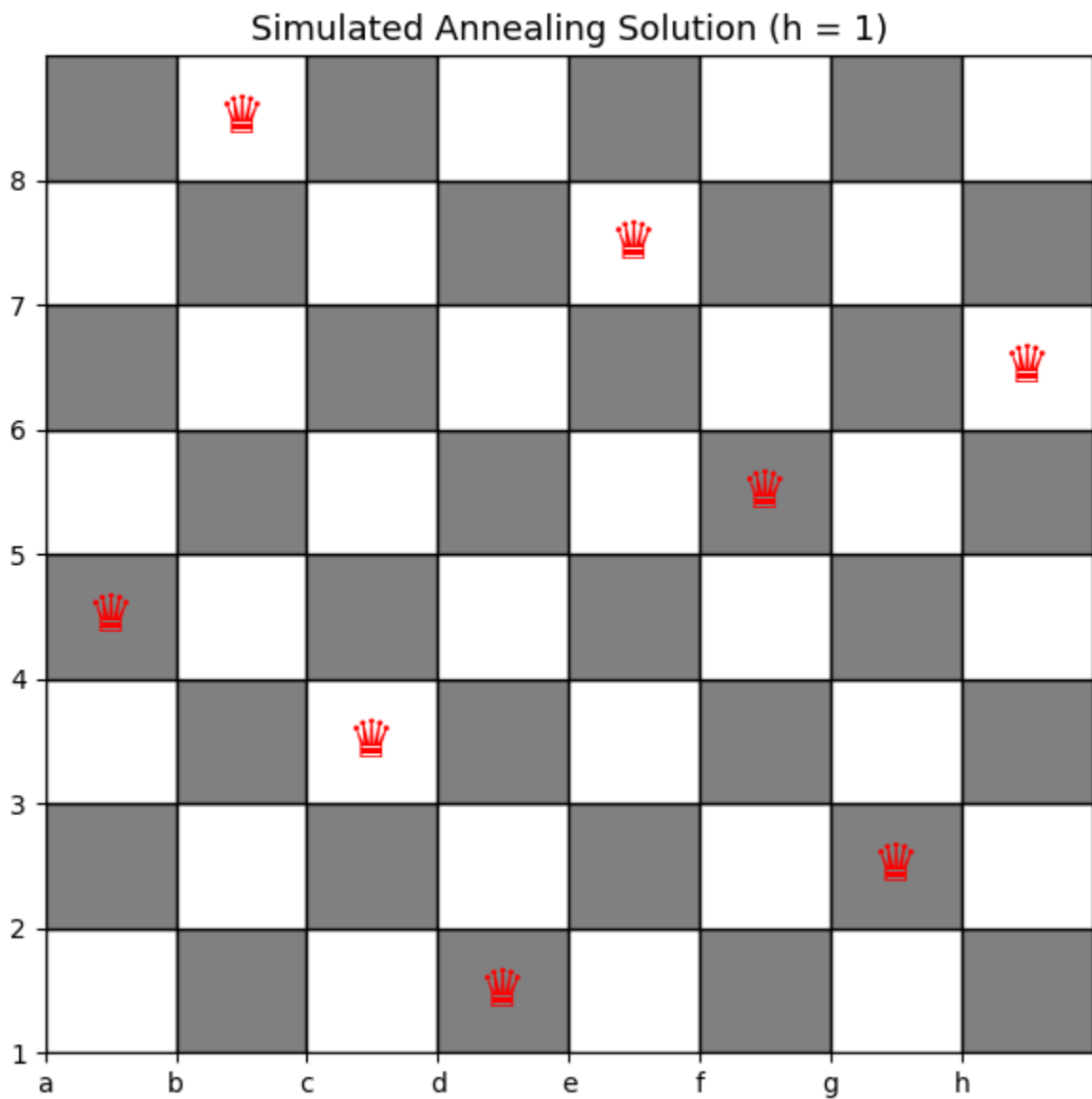
print("Simulated Annealing Solution:")
print("Board:", sa_solution)
print("h value:", h(sa_solution))
```

```
Simulated Annealing Solution:
Board: [3, 7, 2, 0, 6, 4, 1, 5]
h value: 1
```

```
In [9]: plt.figure(figsize=(10, 4))
plt.plot(h_history, color='steelblue')
plt.xlabel('Steps')
plt.ylabel('h (attacking pairs)')
plt.title('Simulated Annealing - h value over time')
plt.grid(True)
plt.tight_layout()
plt.show()
```



```
In [10]: draw_board(sa_solution, title=f'Simulated Annealing Solution (h = {h(sa_solution)})')
```



Output Summary

```
In [11]: cols = ['a','b','c','d','e','f','g','h']

print("=== Hill Climbing ===")
for col in range(8):
    print(f" Queen {col+1}: column {cols[col]}, row {hc_solution[col]+1}")
print(" h =", h(hc_solution))

print()

print("=== Simulated Annealing ===")
for col in range(8):
    print(f" Queen {col+1}: column {cols[col]}, row {sa_solution[col]+1}")
print(" h =", h(sa_solution))
```

```
=== Hill Climbing ===
Queen 1: column a, row 6
Queen 2: column b, row 3
Queen 3: column c, row 1
Queen 4: column d, row 8
Queen 5: column e, row 5
Queen 6: column f, row 2
Queen 7: column g, row 4
Queen 8: column h, row 7
h = 0
```

```
=== Simulated Annealing ===
Queen 1: column a, row 4
Queen 2: column b, row 8
Queen 3: column c, row 3
Queen 4: column d, row 1
Queen 5: column e, row 7
Queen 6: column f, row 5
Queen 7: column g, row 2
Queen 8: column h, row 6
h = 1
```

Short Explanation

	Hill Climbing	Simulated Annealing
Accepts worse moves?	Never	Sometimes
Gets stuck at local min?	Yes	Less likely
Uses temperature?	No	Yes (cools down over time)
Reliability	Lower	Higher

Hill Climbing checks all 56 neighbors and picks the best one. If no neighbor is better it gives up and restarts.

Simulated Annealing picks a random neighbor. If better it always accepts. If worse it accepts with probability $e^{(-\Delta/T)}$. As temperature T drops this probability gets smaller, so it becomes more strict over time.